

EE 446 – TinyML – Spring 2026
Homework 1 – Discussion Questions
Sam Hui

Problem 1 – Part (e): Further Model Size Reduction

Analysis and Comparison

Model	Size (KB)	Accuracy
Base float32	14.07	100%
Dynamic quantization	8.17	100%
Float16 quantization	8.95	100%
INT8 quantization	5.74	100%
Pruned model	14.14	96%
KD student	6.10	98%
KD student + INT8	3.62	98%

The INT8 quantized model (5.74 KB, 100%) was the smallest from parts (b)–(d). The pruned model did not reduce size because TFLite stores sparse weights in the same dense float32 format — the zeros introduced by pruning are still stored as full 32-bit values, so the file size is nearly identical to the base model. The KD student (6.10 KB, 98%) achieved good accuracy with a smaller architecture (32-16-3 vs 64-32-3).

Proposed Strategy

Combine Knowledge Distillation with full INT8 quantization. The two techniques are complementary: KD reduces the number of parameters, and INT8 quantization reduces the bit-width of each parameter. Together they achieve a smaller model than either approach alone.

Implementation

The INT8 quantized KD student model was built by applying TFLiteConverter with Optimize.DEFAULT, TFLITE_BUILTINS_INT8, a representative dataset generator, and int8 input/output types to the already-trained student model. The code is included in the programming notebook (Part e cell).

Results

- **KD Student + INT8 size:** 3.62 KB
- **Test accuracy:** 98% (53/54 correct — same as the unquantized KD student)
- **Size reduction:** 74% smaller than the base float32 model (14.07 KB)

Justification

Yes, further size reduction beyond the smallest model from parts (b)–(d) is possible by combining KD with INT8 quantization. The biggest contributor was applying INT8 quantization to the smaller student architecture: the student already had 67% fewer parameters than the

teacher, and INT8 further compressed each weight from 32 bits to 8 bits. The combined result (3.62 KB, 98%) is smaller and more accurate than the pruned model (14.14 KB, 96%), and smaller than the INT8-quantized teacher (5.74 KB). No significant accuracy was sacrificed.

Problem 2: Exploring Edge Impulse

Question 1 – Learning Blocks on the Free Tier

The following Learning Block types are available on the free-tier Edge Impulse account:

- **Supervised Learning – Classification:** Neural network classifier that maps input features to class probabilities. Example: dense NN for motion/gesture classification from IMU data.
- **Supervised Learning – Regression:** Neural network that predicts a continuous output value. Example: regression NN predicting air quality index from environmental sensor readings.
- **Supervised Learning – Object Detection:** Detects and localizes objects in images using bounding boxes. Example: FOMO (Faster Objects, More Objects) model for person detection on Arduino Nicla Vision.
- **Supervised Learning – Transfer Learning (Images):** Fine-tunes a pre-trained image classification backbone on a custom dataset. Example: MobileNetV2 fine-tuned to classify three types of surface defects.
- **Unsupervised Learning – Anomaly Detection (GMM):** Gaussian Mixture Model that learns the distribution of normal data; samples outside the distribution are flagged as anomalies. Example: vibration anomaly detection on industrial machinery.
- **Unsupervised Learning – Anomaly Detection (K-Means):** K-means clustering of normal operating conditions; samples far from all cluster centroids are flagged. Example: IMU-based anomaly detection on rotating equipment.

Question 2 – Classifier Layers in the Free Tier

The following layer types are available in the Classifier section of free-tier Edge Impulse Studio:

- **Dense:** Fully connected layer; every neuron receives input from all neurons in the previous layer. Used to learn feature combinations and produce final class scores.
- **Flatten:** Converts a multi-dimensional tensor into a 1-D vector so it can be fed into a Dense layer after a convolutional block.
- **Reshape:** Changes the shape of the input tensor without altering its values. Used to prepare data in the format required by downstream layers.
- **Dropout:** Randomly zeroes a fraction of activations during training to reduce overfitting.
- **1D Convolution (Conv1D):** Applies learnable filters along the time/sequence dimension of the input. Designed for time-series and audio data.
- **MaxPooling1D:** Reduces the temporal dimension of a Conv1D output by keeping only the maximum value in each pooling window.

- **Batch Normalization:** Normalizes layer activations to zero mean and unit variance across the mini-batch, stabilizing and accelerating training.
- **LSTM:** Recurrent layer with a gated cell state for learning long-range temporal dependencies in sequential data.

Question 3 – Model Compression Options in Deployment

The following model compression options are available in the Deployment section of the free-tier Edge Impulse Studio:

- **Quantized (int8):** Reduces model weights from float32 to int8 (~4× size reduction) with minimal accuracy loss.
- **Unoptimized (float32):** Baseline model at full floating-point precision — no compression applied.
- **EON Compiler:** Edge Impulse's proprietary inference compiler that generates optimized C++ code for the target device, achieving lower latency and smaller flash usage than TFLite Micro.
- **EON Compiler (RAM optimized):** Variant of the EON Compiler with additional memory layout optimizations to minimize peak SRAM usage during inference.
- **EON Tuner:** AutoML tool that searches over model architectures, DSP settings, and quantization options to find the best model within user-defined latency, flash, and RAM constraints.

Question 4 – Synthetic Data Generation on the Free Tier

Edge Impulse supports synthetic data generation for the following input modalities on the free tier:

- Images:
 - DALL-E (OpenAI) – text-prompted realistic image generation.
 - FLUX (Black Forest Labs) – high-fidelity diffusion-based image synthesis.
- Audio / Speech:
 - Text-to-Speech via ElevenLabs – generates labeled speech samples for keyword spotting across diverse voices.
- Time-Series / Sensor Data:
 - Physics-based simulation – generates synthetic IMU, accelerometer, or environmental sensor signals from simulated physical conditions.

Why synthetic data is important: Real-world labeled data for edge AI is expensive, slow, and sometimes impossible to collect (e.g., rare fault events, private environments). Synthetic data allows rapid dataset augmentation, balancing underrepresented classes, and generating hazardous or privacy-sensitive scenarios safely — all without manual collection or labeling.